

Subject: CS336 Assignment 1, Inference

Date: from 2026/8/24 to 2026/8/27

Contents

Inference	1
Autoregressive Generation	1
Sampling Strategies	2
Generation Loop	3

Inference

推理的本质：无法并行

训练完成后，模型进入推理（Inference）阶段。这里有一个和训练本质不同的约束：训练时，一个长度为 L 的序列可以一次前向并行算出所有 L 个位置的预测（teacher forcing）；而推理时，第 $t+1$ 步的输入依赖第 t 步的输出，无法并行。生成 N 个 Token 就必须老老实实做 N 次前向传播。

Autoregressive Generation

逐 Token 生成

语言模型的生成过程本质上是一个离散时间序列的预测问题：模型不是一次性写出整段文本，而是每次只预测一个 Token，然后把自己刚写下的内容当作新的输入，再预测下一个。

核心逻辑

模型根据已有的 Token 序列 $x_{1:t}$ ，输出下一个 Token x_{t+1} 在整个词表上的概率分布：

$$x_{t+1} \sim P(x \mid x_{1:t})$$

整个流程是一个循环：

1. **前向传播**. 把当前序列喂给模型，取最后一个时间步的 Logits；
2. **采样**. 按某种策略从 Logits 中选出一个 Token x_{t+1} ；
3. **拼接**. 把 x_{t+1} 接到序列末尾，得到 $x_{1:t+1}$ ；
4. **回到 Step 1**，直到生成够 N 个 Token 或遇到结束符。

关键点. 整个序列是靠一次又一次的自我拼接长出来的。模型没有全局规划，它在第 t 步做出的选择会永久地约束第 $t+1$ 步能看到的上下文——这正是自回归模型错误会累积的原因。

上下文窗口限制

Transformer 模型具有固定的上下文窗口长度（context_length，本项目取 256）。推理时生成的序列很容易超过这个上限，因为生成上限 max_new_tokens 和窗口长度是两个独立的量。处理方式是**滑动窗口裁剪**：每次前向之前，只保留序列末尾的 context_length 个 Token。

```
idx_cond = generated[:, -context_length:]
```

这里有两个容易混淆的点：

- **裁剪只影响模型能看到什么，不影响序列本身**. generated 仍在持续增长，被裁掉的 Token 依然在最终输出里，只是不再参与注意力计算。换句话说，模型会“忘记”自己开头写了什么——这解释了长文本生成中常见的后文与前文设定矛盾；
- **为什么必须裁**. 一是注意力的计算量与显存占用随序列长度呈 $O(T^2)$ 增长；二是位置编码（RoPE）在训练时只在 $[0, \text{context_length})$ 范围内见过样本，外推到更长的位置属于训练分布之外的行为，结果不可靠。

工业级实现（如 KV Cache + RoPE 外推 / 位置插值）能把“记住开头”的成本降下来，但作业范围内只要求滑动窗口这一种处理。

Sampling Strategies

从 Logits 到采样

模型直接输出的是 **Logits**——未经归一化的预测分值。最朴素的做法是每次取分值最大的 Token（Greedy Search），但这样生成的文本往往重复且枯燥：Greedy 等价于在每一步都选择概率最高的分支，而语言模型的概率分布是“长尾”的，高频的通用词（如“the”、“and”）会反复胜出，模型一旦进入某个循环就再也出不来。为了生成多样化的文本，我们需要在采样前对 Logits 进行干预。

Temperature（温度缩放）

温度参数 τ 用于调整概率分布的熵（Entropy）。做法是把 Logits 先除以 τ 再过 Softmax：

$$P_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

- $\tau > 1$ ：缩小 Logits 之间的差异，概率分布趋向均匀（High Entropy），生成的文本随机性增加，也更容易出现语法错误；
- $\tau < 1$ ：放大 Logits 之间的差异，高分值 Token 的概率显著增加（Low Entropy），生成的文本更加确定和保守；
- $\tau = 1$ ：不改变原始分布。

本质. 关键在于 Softmax 是指数函数： $\exp(z_i)/\exp(z_j) = \exp(z_i - z_j)$ ，所以 Logits 的差值被 τ 直接缩放了 $\frac{1}{\tau}$ 倍。 $\tau < 1$ 时差值被放大，指数效应随之加剧，分布更尖锐。

两点注意.

- Temperature 不改变 Logits 的排序，只改变分布的平坦程度。所以它无法“换一个候选”，只能改变各候选之间的相对权重；
- $\tau \rightarrow 0$ 的极限就是 Greedy：概率最高的 Token 独占全部质量。因此 Greedy 可以看作 $\tau = 0$ 的特例，实现上直接走 argmax 分支即可（既避免除零，也省一次 Softmax）。

Top-p (Nucleus) Sampling

Temperature 的局限在于它是全局的：整个词表共享同一个 τ 。但模型在不同位置的确定性差别很大——有时它非常确定下一个词是什么，有时它只是在几个合理选项之间犹豫。固定的 τ 无法同时适配这两种情况：调大 τ 会让确定的位置变得胡言乱语，调小 τ 又救不了犹豫的位置。

Top-p 采样（Nucleus Sampling）的思路是动态截断概率分布的尾部：只在累积概率达到 p 的最小 Token 集合中采样。这个集合的大小随分布形状自适应——分布尖锐时集合很小，分布平坦时集合自动变大。相比 Top-k 采样（固定保留概率最高的 k 个），Top-p 的优势正在于这种自适应性：Top-k 用一个固定大小的候选集去应对形状千变万化的分布， k 太小

会切断合理选项， k 太大会放进垃圾 Token.

算法步骤.

1. 排序. 将词表 Logits 按降序排列;
2. 累积. 计算 Softmax 后的累积概率分布 (CDF);
3. 截断. 找到累积概率超过阈值 p 的位置, 将该位置之后的 Token 概率置为 0 (Logit 置为 $-\infty$);
4. 重归一化. 对剩余的 Logits 重新计算 Softmax, 然后采样.

极易写错的实现细节. 第 3 步中, 被丢弃的不应该是“第一个使累积概率超过 p 的 Token”, 而是它之后的所有 Token——因为那个 Token 本身的累积概率还没超过 p , 它理应留在核内. 如果直接照搬“累积概率 $> p$ 就丢弃”, 会平白无故地切掉边界上的一个合法候选, 在候选集本来就很小 (分布尖锐的情况) 影响尤其明显. 实现上通过把掩码整体右移一位来修正:

```
sorted_mask = cumulative_probs > top_p
sorted_mask[..., 1:] = sorted_mask[..., :-1].clone()
sorted_mask[..., 0] = False
```

最后再用 scatter_ 把排序后的 Logits 恢复到原始词表顺序, 把被过滤的位置填成 $-\infty$, 交回调调用方做 Softmax.

Generation Loop

生成循环的实现

把上述组件组装成 TransformerLM.generate(), 单步逻辑为: 滑动窗口裁剪 → 前向传播 (只取最后一个时间步) → 分支 (Greedy 或温度缩放 + Top-p) → 拼接并检查是否结束.

```
idx_cond = generated[:, -self.context_length:]

logits = self.forward(idx_cond)[:, -1, :]

if temperature == 0:
    next_token = torch.argmax(logits, dim=-1, keepdim=True)
else:
    logits = logits / temperature
    if top_p < 1.0:
        logits = self._top_p_filter(logits, top_p)
    probs = softmax(logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)

generated = torch.cat((generated, next_token), dim=1)
if eos_token_id is not None and (next_token == eos_token_id).all():
    break
```

实现要点.

- logits[:, -1, :] 的取法: 前向传播返回的是 (Batch, T, vocab) 的完整张量, 但自回归生成只关心最后一个位置; 中间位置的 Logits 在这一步是无用计算——这正是 KV

Cache 要解决的问题;

- **EOS 提前终止.** 遇到 `<|endoftext|>` 就跳出循环, 避免无意义地生成到 `max_new_tokens`;
- **复杂度.** 当前实现每一步都对完整的前缀重做一次前向, 且注意力本身是 $O(T^2)$, 因此生成 N 个 Token 的总开销约为 $O(N \cdot T^2)$. KV Cache 把历史各层的 Key/Value 缓存下来、每步只计算新 Token, 可以把每步的注意力开销降到 $O(T)$, 是推理优化的第一优先级;
- **temperature == 0 必须单独走 argmax:** 不能靠 “设一个极小的 τ ” 来近似, 因为除法会放大数值误差, 甚至产生 Inf/NaN; 而且 Softmax 后仍需采样, 结果并不确定.